

บทที่ 8 : การประกัน

คุณภาพและการทดสอบ

ซอฟต์แวร์

แนะนำการประกันคุณภาพและการทดสอบ 01

ประเภทการทดสอบซอฟต์แวร์ 02

แนวทางการทดสอบซอฟต์แวร์ 03

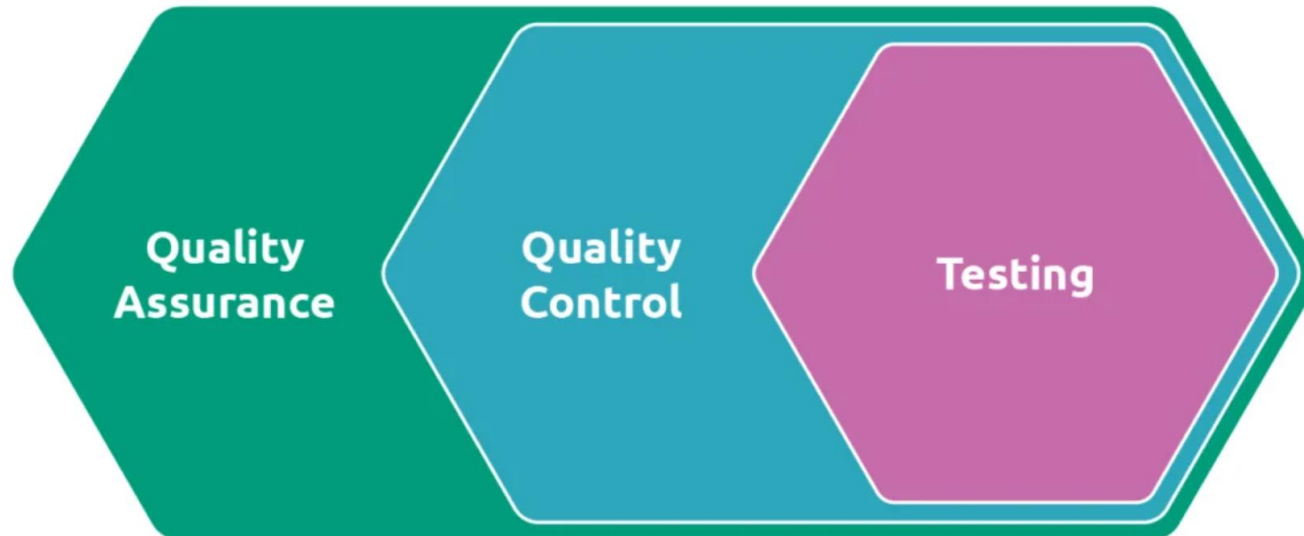
ระดับการทดสอบซอฟต์แวร์ 04

เครื่องมือทดสอบซอฟต์แวร์ 05

1. แนะนำการประกันคุณภาพและการทดสอบซอฟต์แวร์

การประกันคุณภาพซอฟต์แวร์ (Software Quality Assurance) คือ การติดตามกระบวนการทางวิศวกรรมซอฟต์แวร์ (SW Engineering Process) และผลลัพธ์ที่ได้จากกระบวนการ (Work Product) เพื่อให้มั่นใจได้ว่าได้ดำเนินการสอดคล้องกับมาตรฐานที่กำหนดไว้ (เช่น ISO/IEC 15504, CMMI, etc.)

การทดสอบซอฟต์แวร์ (Software Testing) คือ กิจกรรมตรวจหาข้อผิดพลาด (Defect) ที่เกิดขึ้นกับซอฟต์แวร์ เพื่อประเมินคุณภาพของซอฟต์แวร์



ตารางแสดงความแตกต่างระหว่าง QA กับ Software Testing

Aspect	Software Quality Assurance (QA)	Software Testing
Activity Type	Prevention	Detection (Finding bugs)
Focus	Process	Product
Scope	Organization	Specific Product
Life Cycle	Software Development Life Cycle (SDLC)	Software Testing Life Cycle (STLC)
Team	All Software Engineering Teams	Software Testing Team

❏ Software Testing Life Cycle (STLC)

Phase

Description

Work Product

วิเคราะห์ความต้องการ
(Requirement Analysis)

วิเคราะห์ความต้องการที่
ได้รับจาก BA

- รายการ FR และ NFR ที่ต้องการทดสอบ
- สภาพแวดล้อมที่จะทดสอบ (Software/Hardware)

วางแผนการทดสอบ
(Test Planning)

กำหนดกิจกรรม คน
และระยะเวลา

Test Plan

พัฒนาเทสต์เคส
(Test Case Development)

พัฒนาเทสต์เคสโดยระบุ
Input, Output, Test Data ฯลฯ

Test Case

กำหนดสภาพแวดล้อม
(Test Environment Setup)

เตรียม Software/Hardware สำหรับ
การทดสอบ และทำ Smoke Testing

Execution Report

ดำเนินการทดสอบ
(Test Execution)

ดำเนินการทดสอบ
ตามที่ระบุใน Test Case

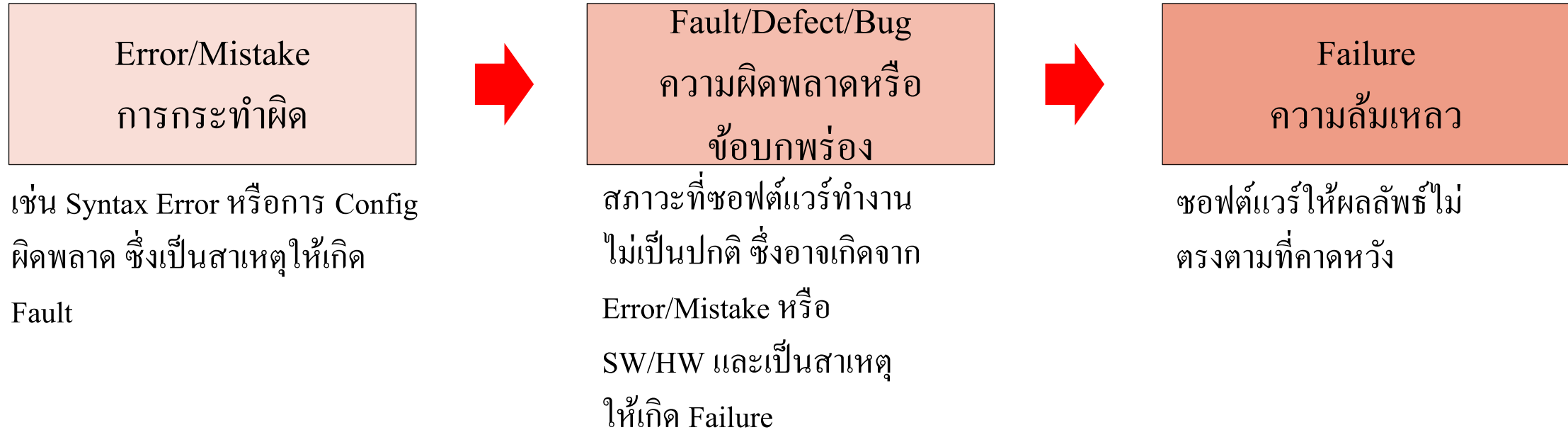
Test Result/
Test Report

ปิดการทดสอบ
(Test Cycle Closure)

ระบุ Best
Practice

Test Closure
Report

❑ คุณลักษณะที่ไม่พึงประสงค์ของซอฟต์แวร์



❑ คุณภาพซอฟต์แวร์ (Software Quality)

คุณลักษณะซอฟต์แวร์	คำอธิบาย
Accuracy	มีฟังก์ชันการทำงานมีความถูกต้องและความสมบูรณ์
Efficiency	มี Response time ที่ดี และไม่ใช้ทรัพยากรเกิดความจำเป็น (CPU, RAM, Storage)
Usability	น่าใช้ ใช้งานง่าย ใช้แล้วอยากกลับมาใช้ใหม่
Security	มีความปลอดภัย เช่น ความปลอดภัยจาก Cross Site Scripting และ SQL Injection
Maintainability	สามารถบำรุงรักษาได้ง่าย รองรับการเปลี่ยนแปลงความต้องการ เช่น - มีการเขียน Comments เพื่ออธิบาย Function, Class หรือ Variable - มีการตั้งชื่อ Function, Class หรือ Variable ให้เข้าใจง่าย - ไม่มีการ Fix ค่าหรือข้อมูลใน Source Code โดยไม่จำเป็น
Portability	รองรับการย้ายการติดตั้งหรือใช้งานระบบในสภาพแวดล้อมใหม่ได้ เช่น เปลี่ยนจาก Server เป็น Cloud, Windows เป็น Linux, หรือ Chrome เป็น Firefox

2. ประเภทการทดสอบซอฟต์แวร์

ประเภทการทดสอบซอฟต์แวร์ (Software Testing Types) ประกอบด้วย การทดสอบแบบแมนนวล (Manual Testing) และการทดสอบอัตโนมัติ (Automated Testing)

การทดสอบแบบแมนนวล (Manual Testing)	การทดสอบอัตโนมัติ (Automated Testing)
- ไม่เหมาะกับการทดสอบซ้ำ ๆ - เหมาะกับ Usability Test - เหมาะกับ Black-Box Testing - มีโอกาสผิดพลาดสูงจาก Human Error	- เหมาะกับการทดสอบซ้ำ ๆ (Regression Testing) - ไม่เหมาะกับการ Usability Test - เหมาะกับ Black-Box และ White-Box Testing - มีโอกาสผิดพลาดต่ำ

3. แนวทางการทดสอบซอฟต์แวร์

แนวทางการทดสอบซอฟต์แวร์ (Software Testing Approach) ประกอบด้วย การทดสอบแบบกล่องดำ (Black-Box Testing) และการทดสอบแบบกล่องขาว (White-Box Testing)

การทดสอบแบบกล่องดำ (Black-Box Testing)	การทดสอบแบบกล่องขาว (White-Box Testing)
■ เป็นการทดสอบอย่างหยาบ (ตรวจสอบแค่ ว่า ถ้า Input กับ Output เป็นไปตามที่กำหนดไว้ ก็ถือว่าทดสอบผ่าน) ■ ผู้ทดสอบไม่จำเป็นต้องมีความรู้ทางด้านโปรแกรมมิ่ง ■ ใช้เวลาน้อยกว่าแบบกล่องขาว	■ เป็นการทดสอบอย่างละเอียด (ตรวจสอบโค้ดในแต่ละบรรทัด ทุกเส้นทางของ IF, While, For) ■ ผู้ทดสอบจำเป็นต้องมีความรู้ทางด้านโปรแกรมมิ่ง ■ ใช้เวลามากกว่าแบบกล่องดำ

❑ Equivalence Partitioning Testing

เป็นเทคนิคการแบ่งข้อมูลทดสอบ (Test Data) สำหรับการทดสอบแบบ Black-Box โดยการแบ่งข้อมูลออกเป็น 2 ส่วนหลัก ๆ คือ ส่วน Valid กับ ส่วน Invalid

■ ส่วน Valid

- ค่าต่ำสุดของข้อมูล
- ค่าสูงสุดของข้อมูล
- ค่าตัวแทนกลุ่มหรือค่าที่ใกล้เคียงกับค่ากลางของข้อมูล

■ ส่วน Invalid

- ค่าที่อยู่นอกขอบของข้อมูล

ตัวอย่าง “Equivalence Partitioning Testing” ในการทดสอบการรับค่าส่วนสูงตั้งแต่ 10-300 ซม.

■ ส่วน Valid

- ค่าต่ำสุดของข้อมูล คือ 10
- ค่าสูงสุดของข้อมูล คือ 300
- ค่าตัวแทนกลุ่ม 150

■ ส่วน Invalid

- ค่าที่อยู่นอกขอบของข้อมูล คือ 5 กับ 350

ตัวอย่าง "Equivalence Partitioning Testing" สำหรับการตัดเกรดโดยมีเกณฑ์ดังนี้

คะแนน 1 – 49 ได้เกรด F

คะแนน 50 – 59 ได้เกรด D

คะแนน 60 – 69 ได้เกรด C

คะแนน 70 – 79 ได้เกรด B

คะแนน 80 – 100 ได้เกรด A

ซึ่งจะต้องแบ่งข้อมูลออกเป็น 3 ค่า สำหรับแต่ละช่วงคะแนน ได้ดังนี้

คะแนน 1 – 49 ได้เกรด F → 1, 30, 49

คะแนน 50 – 59 ได้เกรด D → 50, 54, 59

คะแนน 60 – 69 ได้เกรด C → 60, 67, 69

คะแนน 70 – 79 ได้เกรด B → 70, 76, 79

คะแนน 80 – 100 ได้เกรด A → 80, 84, 100

และข้อมูลที่อยู่นอกขอบคือ -1 และ 150

ส่วนของ Valid

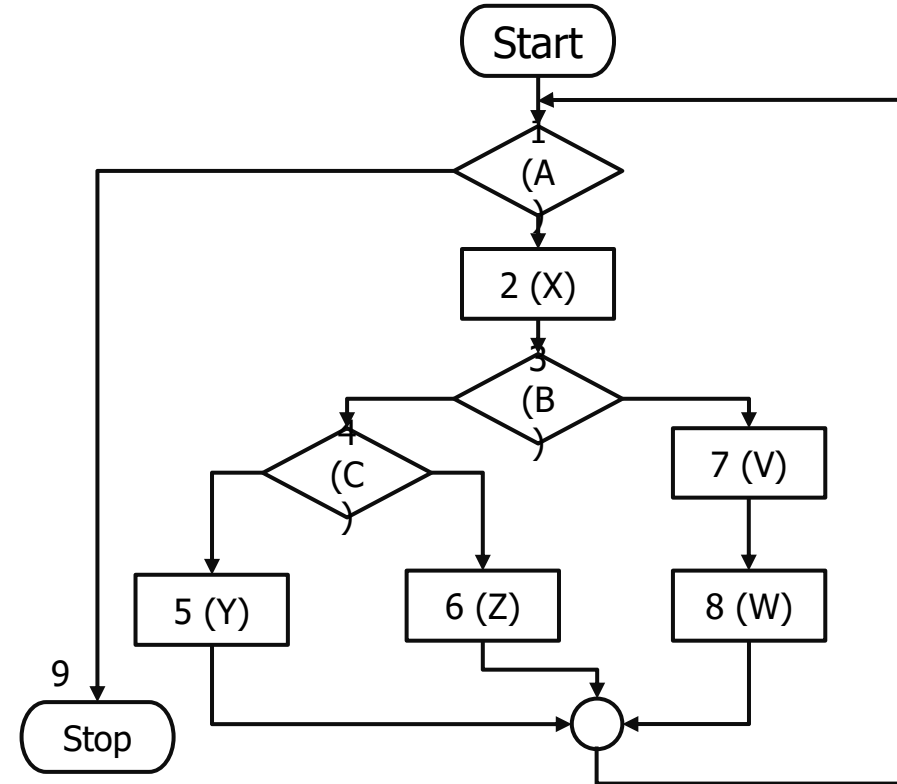
ส่วนของ Invalid

❑ ตัวอย่างการทดสอบแบบกล่องขาว (White-Box)

Source Code

```
function f1(){  
  ① while A {  
    ② process X;  
    ③ if B {  
      ④ if C {  
        ⑤ process Y;  
      }else{  
        ⑥ process Z;  
      }//if  
    }else{  
      ⑦ process V;  
      ⑧ process W;  
    }//end if  
  }//while  
} //function ⑨
```

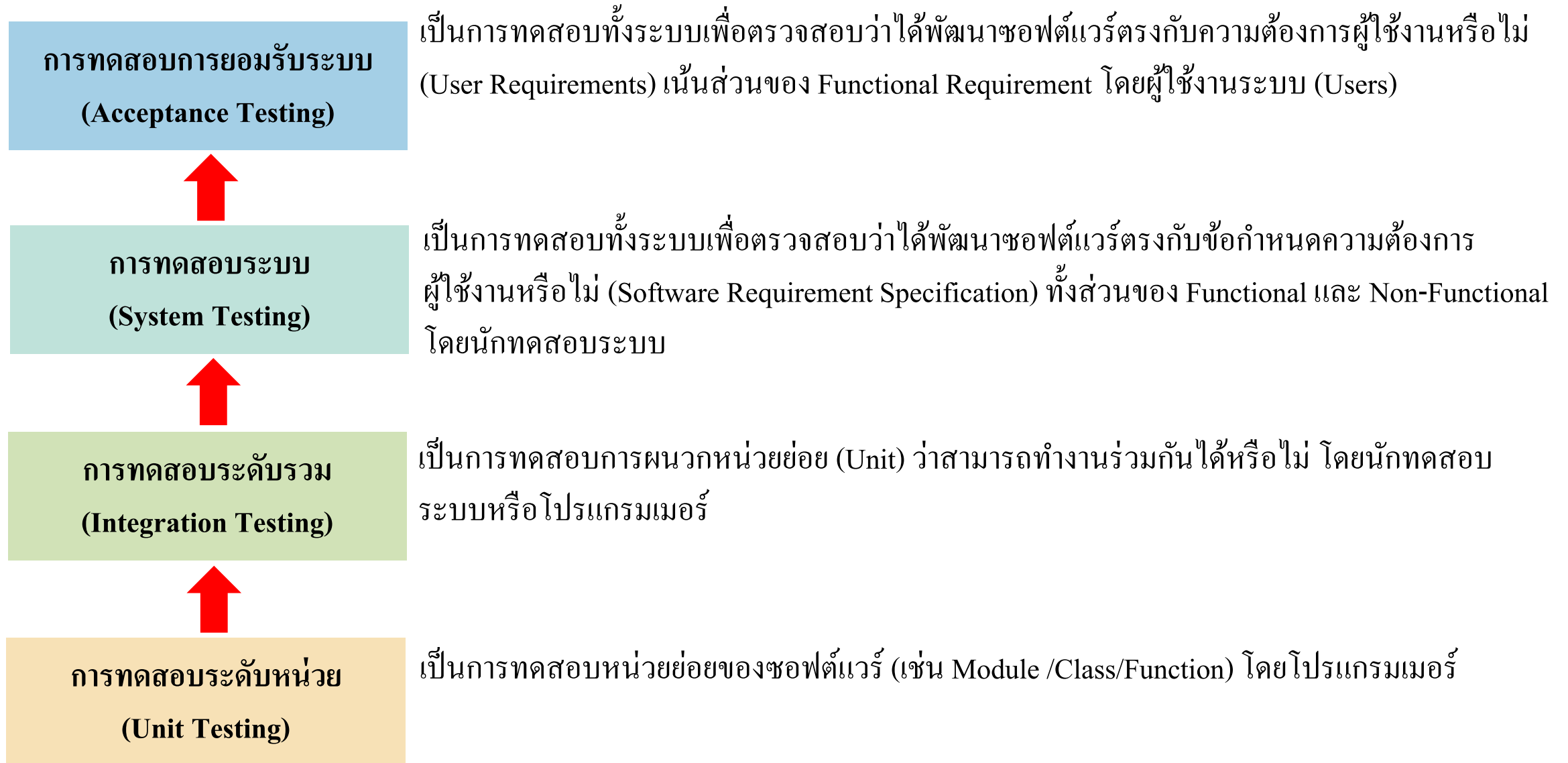
Flow Chat แสดงการทำงาน

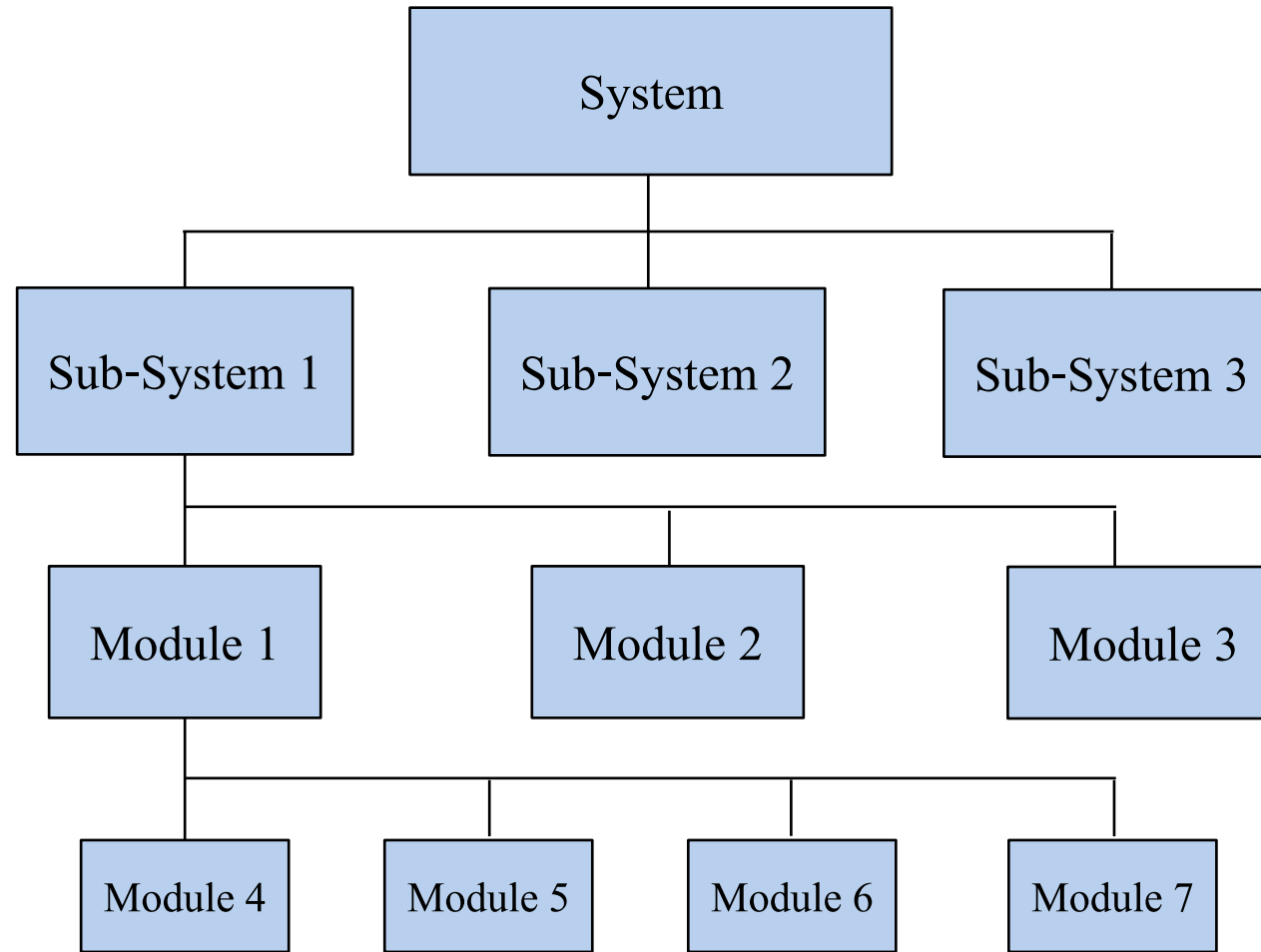


จาก Flow Chart จะพบว่ามี 4 เส้นทางประกอบด้วย

- 1) 1 → 9
- 2) 1 → 2 → 3 → 4 → 5 → 1 → 9
- 3) 1 → 2 → 3 → 4 → 6 → 1 → 9
- 4) 1 → 2 → 3 → 7 → 8 → 1 → 9

4. ระดับการทดสอบซอฟต์แวร์





ภาพแสดงตัวอย่างองค์ประกอบของระบบที่ต้องทดสอบ

❑ การทดสอบระดับหน่วย (Unit Testing)

เป็นการทดสอบหน่วยย่อยที่สุด (Unit) ของซอฟต์แวร์ หน่วยย่อยในที่นี้มักอ้างถึง **Class, Method, หรือ Function** และผู้ที่ทำการทดสอบในระดับนี้คือโปรแกรมเมอร์ ซึ่งมีวิธีการทดสอบระดับหน่วย ดังนี้

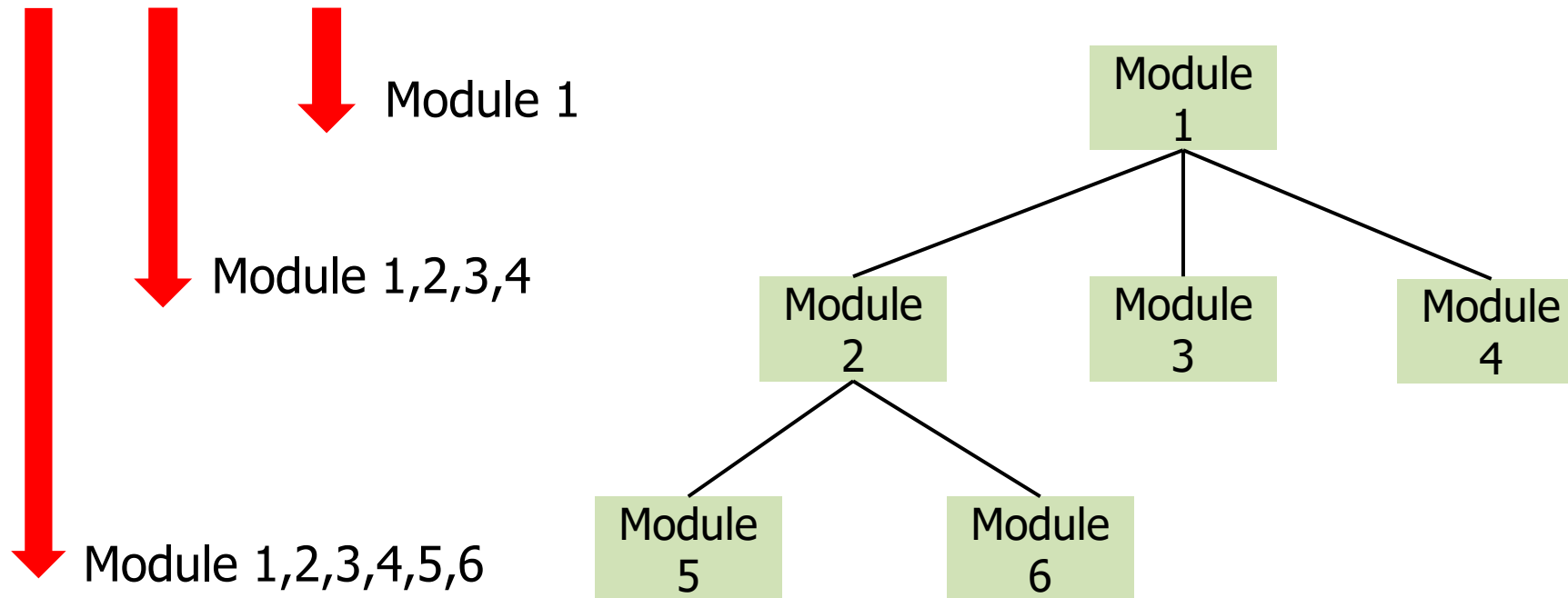
❑ การทดสอบระดับรวมหน่วย (Integration Testing)

เป็นการทดสอบการประสานกันของส่วนประกอบย่อยเพื่อค้นหาข้อผิดพลาดที่เกิดขึ้น ซึ่งมีวิธีการรวมหน่วย ดังนี้

- การทดสอบแบบรวมหน่วยจากบนลงล่าง (Top-Down)
- การทดสอบแบบรวมหน่วยจากล่างขึ้นบน (Bottom-Up)

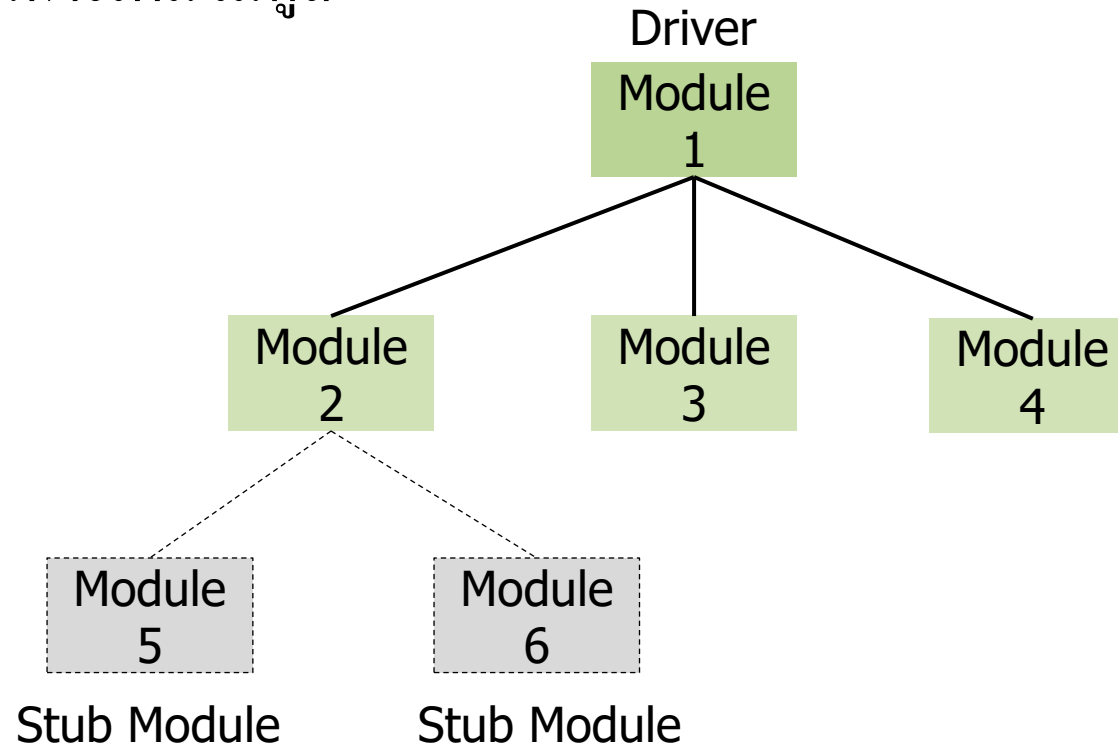
การทดสอบแบบรวมหน่วยจากบนลงล่าง (Top-Down)

- รวมทีละโมดูลจากบนลงล่าง
- โมดูลระดับบนจะเรียกใช้โมดูลระดับล่าง



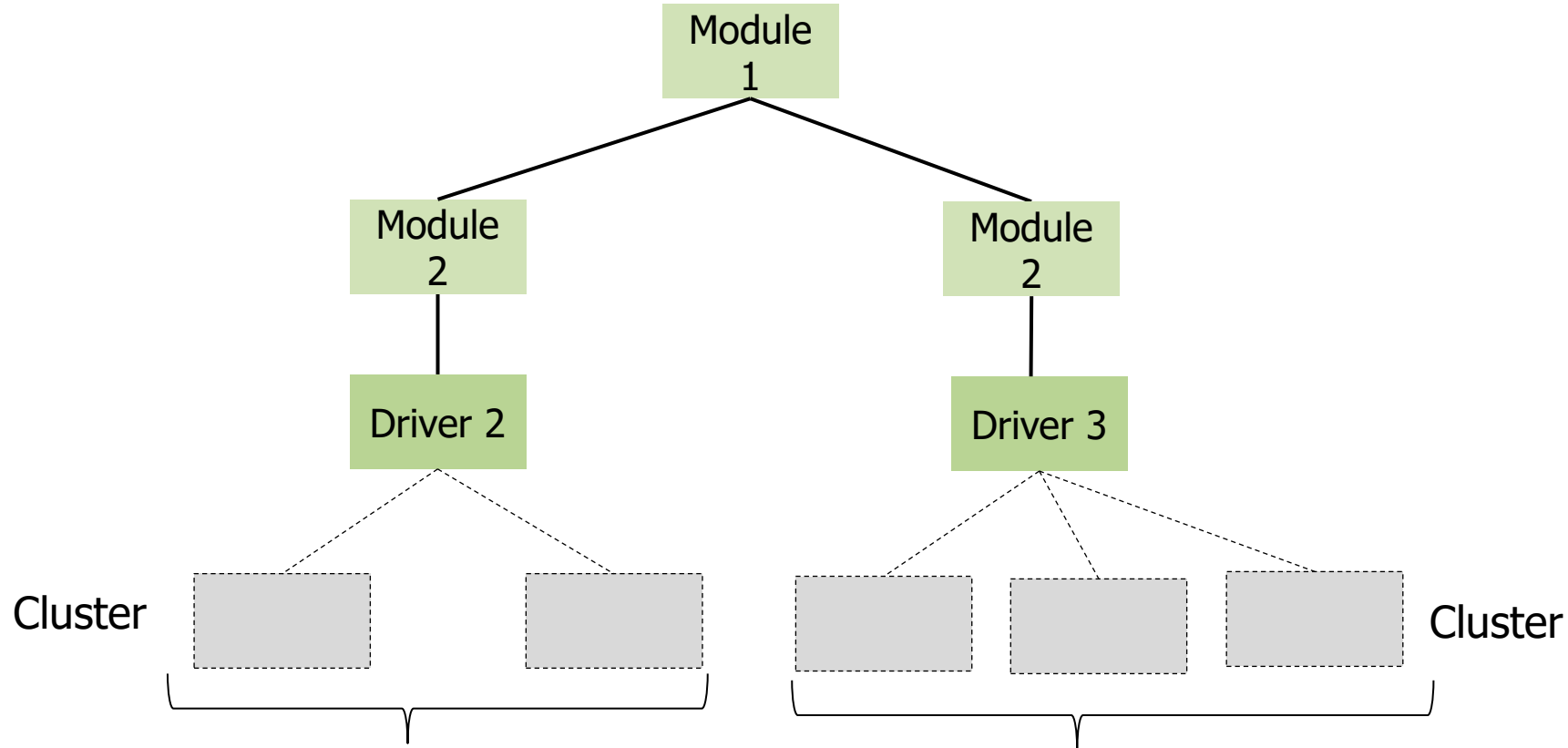
ในการทดสอบแบบรวมหน่วยจากบนลงล่างมีหลักการดังนี้

- ต้องมีโมดูลหลักเรียกว่า “**Driver**”
- หากโมดูลที่ทดสอบต้องการ โมดูลย่อยจึงจะสมบูรณ์ แต่ขณะนั้นยังไม่มีโมดูลย่อย ให้ทำการสร้างโมดูลแทนเรียกว่า “**Stub Module**”
- ทดสอบทุกครั้งที่มีการเพิ่มโมดูล



การทดสอบแบบรวมหน่วยจากล่างขึ้นบน (Bottom-Up)

- รวมโมดูลทีละ Cluster จากล่างขึ้นบน
- การแบ่ง Cluster มักจะแบ่งตามหน้าที่ของซอฟต์แวร์
- สร้าง Driver เพื่อทำหน้าที่ส่งและแสดงข้อมูลจาก Cluster เมื่อทดสอบแล้วจึงถอด Driver ออก จากนั้นก็ทำการทดสอบ Cluster ใหม่กับ Cluster เก่า ซึ่งทำแบบนี้ไปเรื่อย ๆ จนครบทุก Cluster



การทดสอบระบบ (System Testing)

เป็นการทดสอบหลังจาก Modules ต่าง ๆ ถูกรวมเข้าด้วยกันแล้ว เพื่อตรวจสอบความสอดคล้องกันกับข้อกำหนดความต้องการซอฟต์แวร์/ระบบ (Software Requirement Specification) ซึ่งทดสอบโดยทีมทดสอบ (Tester) ภายใต้สภาวะจำลอง ในการทดสอบระบบนอกจากต้องทดสอบตามฟังก์ชันการทำงานแล้วต้องทดสอบในส่วนที่ไม่ใช่ฟังก์ชันการทำงานด้วย ดังนี้

- **Usability Testing** : เป็นการทดสอบถึงความน่าใช้และการใช้งานง่ายของระบบ (UX/UI))
- **Performance/Efficiency Testing** : การทดสอบประสิทธิภาพของระบบ (ดูว่า เว็บแต่ละหน้า โหลดเร็วหรือไม่, ระบบรองรับการเข้าใช้งานพร้อมกันได้กี่คน)
- **Stress Testing** : เป็นการทดสอบความตึงเครียด ถือว่าเป็น subset ของ Performance Testing แต่เป็นการทดสอบภายใต้ความต้องการทรัพยากร (CPU, RAM, Disk) ที่สูงมาก ๆ
- **Security Testing** : เป็นการทดสอบความปลอดภัยของระบบ เช่น การตรวจสอบการบุกรุกแบบ Cross-Site Scripting, แบบ SQL Injection, หรือการ Login
- **Portability Testing** : เป็นการทดสอบความสามารถในการย้ายการติดตั้งระบบหรือใช้งานระบบในสภาพแวดล้อมใหม่ เช่น เปลี่ยนจาก Server เป็น Cloud, Windows เป็น Linux, หรือ Chrome เป็น Firefox
- **Recovery Testing** : เป็นการทดสอบการกู้คืนระบบ เพื่อพิสูจน์ว่าหากเกิดความเสียหายต่อซอฟต์แวร์หรือข้อมูล ระบบจะสามารถกลับมาใช้งานได้ปกติ
- **Documentation Testing** : เป็นการตรวจสอบความถูกต้องและความสอดคล้องกันของเอกสาร

การทดสอบการยอมรับระบบ (Acceptance Testing)

เป็นการทดสอบเพื่อตรวจสอบว่าได้พัฒนาซอฟต์แวร์ตรงกับความต้องการผู้ใช้งานหรือไม่ (User Requirements) เน้นส่วนของ Functional Requirement เป็นการทดสอบโดยผู้ใช้งานระบบ (Users)

การทดสอบอัลฟา (Alpha Testing)

- ทดสอบ ณ สถานที่ผลิตซอฟต์แวร์ภายใต้สถานการณ์จำลอง
- ทดสอบระบบโดยผู้ใช้งานระบบ แต่มีทีมพัฒนาคอยสังเกตการณ์และบันทึกข้อผิดพลาดที่เกิดขึ้น

การทดสอบเบต้า (Beta Testing)

- ทดสอบหลังการทดสอบอัลฟา
- ทดสอบ ณ สถานที่ใช้งานจริงภายใต้สถานการณ์จริง
- ทดสอบโดยผู้ใช้งาน ไม่มีทีมพัฒนาของสังเกตการณ์ ดังนั้นผู้ใช้งานต้องทำการบันทึกข้อผิดพลาดแล้วส่งกลับไปยังทีมพัฒนา

5. เครื่องมือทดสอบซอฟต์แวร์

ในการทดสอบซอฟต์แวร์จำเป็นต้องมีเครื่องมือสำหรับการทดสอบ โดยเฉพาะซอฟต์แวร์ขนาดใหญ่ซึ่งต้องใช้เวลาในการทดสอบยาวนาน



<https://phpunit.de/>

เป็นเครื่องมือในการทดสอบ Unit Testing สำหรับโค้ดภาษา PHP



<https://www.getpostman.com/>

เป็นเครื่องมือในการทดสอบ API ทั้งแบบ REST และแบบ SOAP



<https://jmeter.apache.org/>

เป็นเครื่องมือในการทดสอบประสิทธิภาพของ Web Application และ API



Selenium

เป็น Automation Test Framework ที่ได้รับความนิยมอย่างสูง

<https://www.selenium.dev/>



เป็น Automation Test Framework ที่ได้รับความนิยมอย่างสูง
เช่นกัน

<https://robotframework.org/>

Do you have any questions?

